PERINKPHERAL

Application platform
Datasheet and development guide

Display	4.2" e-paper, 400 x 300 pixels, 1 bit
Input	4 buttons
Application	WebAssembly module and manifest
Language	C, no standard library
Interface	one header, sdk/host_api.h, ABI 2
Memory	64 KB per application
Build and upload	about 10 s, no reflashing

I Datasheet: parameters and execution model

II Development guide: package, screen, input, storage, network

III Reference: host API and SDK library

revision 2, 2026-11-14 / host API of sdk/host_api.h, abi 2 / preliminary: the host API is not frozen before the first release

github.com/GrakovNe/perinkpheral / MIT

Contents

Part I. Datasheet 4

1. Overview 4
   - Features 4
   - How an application runs 4

2. Characteristics 4
   - Display 4
   - Input 5
   - Sandbox 5
   - Storage 5
   - Network 6
   - Time and background work 6
   - Text 6

3. Execution model 7
   - Application lifecycle 7
   - Handler guarantees 7
   - State 7
   - Navigation and refresh 7

Part II. Development guide 9

4. A first application 9
   - Requirements 9
   - Create, build, run 9
   - Sample application 10
   - Build report 10

5. Package 11
   - Manifest 11
   - Polls 12
   - Settings 12

6. Screen 13
   - Surface and colour 13
   - Fonts 13
   - Text beyond ASCII 13
   - Pictures and icons 13
   - Kit 15
   - Requesting a refresh 16

7. Buttons 16
   - BACK 16
   - Lists, pickers, dates 17

8. Storage 17
   - Store 17
   - Files 17

9. Network 17
   - Values extracted by the host 18
   - Streamed answers 18
   - Requests from code 18
   - Failed requests 19

10. Background work 19
Notifications 20
Alarms, timers 20
11. Time 20
12. Languages 20
13. Design rules 21
Screen content 21
Memory 21
State and time 21
Interaction and data 22
14. Tools and debugging 22
Common problems 22
15. Publishing 23
Part III. Reference 24
16. Return codes 24
17. Exports 24
18. Drawing 25
19. Kit 25
20. Navigation, storage and network 26
21. Time 27
22. System API 27
23. SDK library 28
app_text.h: short strings without printf 28
app_values.h: what the host picked 28
app_ui.h: what the buttons do on the kit's screens 29
app_alloc.h, json_pick.h, and what is generated 29
24. Limits 29

Part I. Datasheet

1 Overview

perINKpheral is a desk display: a 4.2" black-and-white e-paper panel with four buttons, built on an ESP32-C6. It shows information that is read at a glance: the time, the weather, the next meeting, the state of a home.

The firmware is called the *host*. The host provides the menu and the settings. Everything else on the screen is an *application*. Applications are installed from a catalog without reflashing the device.

An application consists of a WebAssembly module and a manifest. The module is written in C against a single header, `sdk/host_api.h`. The host runs it in an interpreter (wasm3).

The host's own pages, Settings and About, are applications built with the same header.

Features

- Display: 400 x 300 pixels, 1 bit, dithered greys, partial refresh in about 0.5 s
- Input: four buttons; short press, long press, key down and key up events
- Sandbox: 64 KB of private memory; network access through the host only
- Network: HTTP and HTTPS requests performed by the host, as scheduled polls or on request from the code
- Data: the host extracts values from JSON answers; any answer can be streamed in 1 KB pieces
- Background work: answers, alarms and device events are delivered while the application is off the screen
- Storage: a key-value store and files, kept across restarts and reinstalls
- Design system: lists, notices, pickers, cells and a button row, drawn by the host
- Notifications: shown by the host over any screen
- Settings: declared in the manifest, edited on the device's web page
- Localization: strings per language, loaded by the host; Latin with diacritics, Cyrillic and Greek
- Tools: build, check and measure an application; control a device over the network

How an application runs

An application is a set of exported handlers with fixed names. The host calls a handler when an event occurs: a button press, a timer, incoming data, a request to draw.

A handler updates the application's state and requests a redraw. The host calls `app_on_draw()` when it needs a frame. The host decides when and how the panel is refreshed.

One application is loaded at a time. The host unloads it when it leaves the screen. Persistent state is kept in the application's store; global variables live until the unload.

2 Characteristics

Display

Parameter	Value	Notes
Panel	4.2" e-paper, SSD1683	No backlight. The image is retained without power
Resolution	400 x 300 px	About 120 px per inch

Parameter	Value	Notes
Application surface	400 x 260 px	Below the host header. 400 x 300 with `"fullscreen": true`
Depth	1 bit	17 grey levels by fixed 4 x 4 dithering; for filled areas
Partial refresh	0.46 - 0.53 s	Only the changed rectangle is sent
Full refresh	2.2 s (2.7 s)	Fast waveform (default waveform). The panel flashes
Full refresh interval	every 5 partial refreshes	User setting. Removes ghosting left by partial refreshes
Refresh after a button press	immediate	
Refresh without a button press	max. 1 per 60 s	User setting. The panel manufacturer recommends 180 s
Frame sequence rate	about 1.5 frames/s	`HOST_REDRAW_NO_BLINK`. Ghosting accumulates until the sequence ends

Input

Parameter	Value	Notes
Buttons	4	Order on the device: UP, OK, DOWN, BACK
Events per button	4	Key down, key up, pressed, long pressed
Long press	600 ms	Reported while the button is held
Reserved	long press of BACK	Handled by the host: clears the stack and opens the menu

Sandbox

Parameter	Value	Notes
Linear memory	64 KB, fixed	One wasm page: 8 KB stack, static data, remainder for `app_alloc()`. Fixed size
Module (`app.wasm`)	max. 48 KB	Executed from flash
Interpreter memory	64 KB	About 11 KB base plus 4 bytes per byte of executed code
Executable code, all paths run	about 13 KB	Derived from the interpreter memory. `mkapp` warns when the estimate is exceeded
Handler time limit	3 s	On expiry the handler is stopped and the application terminated
Loaded instances	1	Across all applications
Floating point	integer only	The build rejects a module with a float operation
C library	`sdk/lib`	Strings, numbers, an arena allocator, the `memcpy` family. The standard C library is absent
Installed applications	6	Per device. Catalog size is unlimited

Storage

Parameter	Value	Notes
Store key	1 - 15 characters	Characters `!` to `~`
Store value	1024 bytes	

Parameter	Value	Notes
Store total	4096 bytes	Per application. Kept after removal and restored on reinstall
Files total	64 KB	Per application: packaged files and files written by the code. Each file counts as whole 4 KB blocks (16 blocks). Guaranteed for every installed application
File name	1 - 23 characters	`a–z 0–9 . – _`; the first character is a letter, digit, – or _

Network

Parameter	Value	Notes
Transport	HTTP, HTTPS	Performed by the host
HTTPS trust	browser root set	Self-signed certificates are rejected. Plain `http` is available for home networks
Hosts for requests from code	4	Listed in the manifest, or taken from a `url` or `link` setting. Polls use the address in their `url`
Polls per manifest	4	Interval 10 s to 1 day; minimum 300 s on battery
Concurrent requests	1	Per device. Queue: 2 more
Request headers / body	1023 / 512 bytes	Up to 4 header lines
Answer with extracted values	256 KB	JSON; up to 16 paths; value up to 63 bytes
Streamed answer	4 MB	Pieces of up to 1024 bytes; about 90 KB/s over plain HTTP
Redirects	3	`GET` only. Requests from code stay within the hosts list
HTTPS request time	about 6 s	Mostly the TLS handshake

Time and background work

Parameter	Value	Notes
Timers	4	One-shot, milliseconds. Active while the application is on the screen
Alarms	4	Relative, absolute, or repeating every 60 s to 1 day. Active on and off the screen. Cleared by a restart
Device events (intents)	3	Started, time changed, network up
Queued notifications	4	Per device
Screen takeover	max. 1 per 30 s	`request_focus()`; granted by the host
Clock	Unix seconds	Set from the network. 0 until first set. The time zone is a host setting

Text

Parameter	Value	Notes
Encoding	UTF-8	All text. Passed as pointer and length
Built-in glyphs	printable ASCII, °	
Glyph packs	3	Latin with diacritics, Cyrillic, Greek. Installed by the user on the device's web page
Fallback	ASCII transliteration	"Moskva", "Strasse", "Athina"; otherwise ?

Parameter	Value	Notes
Text per drawing call	127 bytes	Longer text is cut between characters
Font sizes	6	Capital height 8 to 80 px. Two typefaces, selected by the user
Languages	27	English built in; other languages as tables loaded by the host (section 12)

3 Execution model

The host loads an application to call a handler and can unload it after the call.

Application lifecycle

Event	Instance	Handlers, in order
The user opens the application	Loaded; stays while it has the screen	`app_init, app_on_focus, app_on_draw ... app_on_blur, app_on_terminate`
An answer, alarm, intent or settings change while on the screen	Same instance	`app_on_data / app_on_values / app_on_alarm / app_on_intent / app_on_settings`
The same while off the screen	Loaded for the call, unloaded after it. The current screen is suspended and restored	`app_init, the handler, app_on_terminate`
`request_focus()` granted by the host	Stays, on top of the stack	`... app_on_focus, app_on_draw`
The application leaves the screen: BACK, the menu, another application on top	Unloaded	`app_on_blur, app_on_terminate`

Handler guarantees

- Handlers run one at a time, each to completion. Locking is unnecessary.
- A handler has 3 seconds. A handler that runs longer, or traps (for example, on an invalid pointer), is stopped. The application is then terminated and the screen shows an error. The host keeps running.
- Handlers return `void`. A redraw, the screen or an exit is obtained with a request. A request takes effect after the handler returns.
- The host calls `app_init()` before any other handler of an instance.
- Button events arrive in order and are all delivered. Pending timers with the same number are merged into one. The pieces of one answer arrive in order, in consecutive calls.
- Background delivery of an answer can be interrupted. If the user presses a button, the host unloads the instance and drops the remaining pieces. The application receives `app_on_terminate()` only. The host repeats the poll after one minute.

State

Global variables live until the application is unloaded. Unloading happens every time the application leaves the screen and after every background call.

State is loaded in `app_init()` and saved in `app_on_terminate()`, the last call every instance receives. A value that must survive a power loss is saved immediately after it changes.

Timestamps that stay valid across restarts: `time_now()`, or `time_uptime_ms()` together with `time_boot_id()`. The boot id changes after every restart.

Navigation and refresh

Navigation is a stack. The menu is at the bottom. An application opened from the menu is placed on top. A notification, a question from the host or an application that requested the screen is placed on top of the current screen.

A short press of BACK removes the top entry. A long press of BACK clears the stack and opens the menu.

Only the top entry is loaded. An application lower in the stack is unloaded and restores its state when it returns to the top.

The host controls the panel. It compares each new frame with the frame on the panel and sends only the changed area. It postpones a refresh that was not caused by a button press if the previous one was too recent.

The panel always shows a complete frame. Refresh timing is decided by the host.

Part II. Development guide

4 A first application

Requirements

- **wasi-sdk** 34 or newer. It provides clang for the wasm32 target. Set `WASI_SDK_PATH` to its location.
- **Python 3**. The tools use the standard library only.
- A checkout of the **perinkpheral** repository. It contains `sdk/` and `tools/`.
- A **device** on the same network, running a firmware with the development tools enabled (the default). The device shows its address on the second page of About. Set `PERINKPHERAL_HOST` to this address.

Create, build, run

```
tools/devtool/devtool.py new my-app --title "My app"     # create a folder from the SDK sample
tools/devtool/devtool.py run my-app                      # build, upload, restart into the application
tools/devtool/devtool.py serve my-app                    # local page: live screen, buttons, log, Run
```

`run` builds the application, uploads it and restarts the device into it. This takes about ten seconds. The firmware is not reflashed.

`serve` starts a local web page with the live screen, the four buttons, the device log and a Run button. The arrow keys, Enter and Backspace act as the buttons.

`new` also creates a `CMakeLists.txt` in the application's folder. Open this folder in an IDE that supports CMake, such as CLion or VS Code.

The IDE resolves the SDK headers and the generated `strings.h` and `pictures.h`. The project has three build targets: `my-app` (build), `my-app-run` (build and run on the device) and `my-app-serve` (the live page).

Set `WASI_SDK_PATH` and `PERINKPHERAL_HOST` in the CMake environment of the IDE.

Sample application

```
#include "app_text.h"
#include "host_api.h"

#define MARGIN 16

static const char PRESSES_KEY[] = "presses";
#define PRESSES_KEY_LENGTH ((int32_t)sizeof PRESSES_KEY - 1)

static int32_t presses;

static void draw_line(int32_t line, const char *text)
{
    int32_t line_height = gfx_font_height(HOST_FONT_TEXT);
    gfx_text(MARGIN, MARGIN + line * line_height, HOST_FONT_TEXT, text, app_text_length(text));
}

void app_init(void)                 /* load the state from the store */
{
    kv_get(PRESSES_KEY, PRESSES_KEY_LENGTH, &presses, sizeof presses);
}

void app_on_terminate(void)         /* save the state before the unload */
{
    kv_set(PRESSES_KEY, PRESSES_KEY_LENGTH, &presses, sizeof presses);
}

void app_on_button(int32_t event)   /* update the state, request a redraw */
{
    switch (event) {
    case BTN_UP_PRESSED:
    case BTN_OK_PRESSED:
    case BTN_DOWN_PRESSED:
        presses++;
        request_redraw(HOST_REDRAW_NORMAL);
        break;
    }
}

void app_on_draw(void)              /* draw the whole frame from the state */
{
    char       counter_buffer[32];
    app_text_t counter;
    app_text_start(&counter, counter_buffer, sizeof counter_buffer);
    app_text_add(&counter, "Presses: ");
    app_text_add_number(&counter, presses);

    draw_line(0, "Hello from wasm");
    draw_line(2, counter_buffer);
}
```

BACK is handled by the host here: a short press of BACK closes an application that has no `app_on_back()` export.

Build report

```
tools/mkapp/mkapp.py my-app --out build/my-app

build/my-app/app.wasm: 811 bytes, 5 host calls
  module  0.8 KB of 48.0 KB
  code    0.5 KB; running it takes about 13.0 KB of the interpreter's 64.0 KB
  memory  9.1 KB of 64.0 KB: 8.0 KB of stack, 1.1 KB of data; 54.9 KB left for app_alloc()
```

Line	Measures	When exceeded
`module`	File size: code and initial data. Executed from flash	Move pictures and tables to `files/`. Use the kit instead of custom drawing
`code`	Code size and the estimated interpreter memory needed to run all of it. **The scarcest resource**: one byte of code uses four bytes of memory	See "Memory" in section 13
`memory`	The 64 KB page: stack, static data, and the remainder for the arena	Allocate large buffers with `app_alloc()` under a mark. Read files in parts

The build also fails in these cases:

- the module imports a function that `host_api.h` does not declare;
- the code contains a floating-point operation (the report names it);
- the code calls a `sys_*` function and the manifest lacks `"system": true`;
- the manifest exceeds a limit;
- a translation does not match `en.json` (section 12).

5 Package

```
my-app/
  manifest.json      identity, polls, settings
  my-app.c           source code: one or more .c files
  strings/           optional: en.json, ru.json ...; one file per language    (section 12)
  pictures/          optional: PNG, PBM, or text rows of '#' and '.'          (section 6)
  files/             optional: data files, read with file_read()
```

`mkapp` builds the package from this folder. It compiles the sources with the SDK library, generates `strings.h` from `strings/` and `pictures.h` from `pictures/`, and checks all limits. The package contains `app.wasm`, `manifest.json`, the files and one string table per language.

Manifest

```
{
  "name": "plex",
  "title": "Plex Now Playing",
  "version": "0.1.0",
  "abi": 2,
  "hosts": ["{{server}}"],
  "polls": [
    { "url": "{{server}}/status/sessions", "every": "{{every}}",
      "headers": { "Accept": "application/json", "X-Plex-Token": "{{token}}" },
      "pick": ["MediaContainer.Metadata[0].title", "MediaContainer.Metadata[0].grandparentTitle"] }
  ],
  "settings": [
    { "key": "server", "type": "url",    "title": "Server" },
    { "key": "token",  "type": "secret", "title": "Plex token" },
    { "key": "every",  "type": "choice", "title": "Refresh",
      "options": ["15 s", "1 min", "5 min"], "default": "1 min" }
  ],
  "intents": ["started"],
  "icon": ["...........", ".#########.", ".#.......#.", ".#..#....#.", ".#..###..#.", ".#..####.#.",
           ".#..###..#.", ".#..#....#.", ".#.......#.", ".#########.", "..........."]
}
```

Field	Format	Description
`name`	1 - 13 of `a-z 0-9 - _`; equal to the folder name	Identifier: device folder, catalog key, store namespace. A changed name is a different application
`title`	1 - 23 characters	Display name in the menu and on the web page. Translatable
`version`	up to 15 bytes	Set by the developer. A different version in a catalog is offered as an update
`abi`	2	Host API version of the build. A mismatch is rejected at install
`fullscreen`	true / false (default)	false: host header on top, surface 400 x 260. true: surface 400 x 300
`icon`	11 rows of 11 of # and .	Shown in the menu, on the web page and in catalogs. Optional
`polls`	up to 4	Scheduled requests performed by the host (below)
`hosts`	up to 4, each up to 63 bytes	Destinations allowed for `http_request()`: a name, a name with a port, or `{{key}}` of a `url` or `link` setting. Required only for requests from code; polls are exempt
`settings`	up to 8	Settings form on the device's web page (below)

Field	Format	Description
`intents`	`started`, `time_changed`, `network_up`	Device events that load the application. An unknown value fails the build
`system`	true / false (default)	Enables the `sys_*` calls. The web page warns the user before installation
`companion`	`https` URL	Developer's page opened from the settings form, for setup that a form cannot express

The host ignores unknown keys. `mkapp` validates the keys listed above.

Polls

Field	Format	Description
`url`	Up to 511 bytes. `http` or `https`; or starting with `{{key}}` of a `url` setting; or exactly `{{key}}` of a `link` setting	`{{key}}` of any setting is replaced by the stored value, percent-encoded: every character except letters, digits and `- . _ ~` becomes %XX, **including `/`**. One setting is one path segment: `owner/name` requires two settings, `/repos/{{owner}}/{{name}}`. A leading `url` setting and a `link` setting are inserted unchanged. A poll with an empty setting in its URL or headers fails with `HOST_FAILED_NOT_READY`; a token in a header is therefore required
`every`	10 to 86400 seconds, or `"{{key}}"` of a setting	Interval. With a setting, the stored text is a number and a unit; the unit is identified by its first letter: `s`, `m`, h, d ("30 s", "15 min", "1 h", "1 day"). A number alone is seconds. The host applies the user's choice
`headers`	up to 4; a value up to 255 bytes	`{{key}}` is supported in values. Used to send a token
`pick`	up to 16 paths of up to 63 bytes	JSON paths, for example `current.temperature_2m`, `daily.code[0]`. The host extracts the values. Without `pick` the answer is streamed

Polls are numbered from 0 in the order of the list. Handlers receive this number.

Settings

Settings are declared in the manifest. The device's web page builds the settings form from the manifest alone. Each value is stored as text in the application's store, under the key of the setting.

The application reads a setting with `kv_get()` and the key of the setting.

When the user saves the form, the host calls `app_on_settings()` and repeats all polls immediately.

Type	Form field	Stored value
`text`	Text line	The text, up to 255 bytes
`number`	Number	Digits, optional sign and decimal point
`choice`	Selection from up to 8 `options`	The option text, 1 - 23 characters. Translations add labels; the stored value is the option
`toggle`	Checkbox	`"1"` or `"0"`; empty until first saved
`secret`	Password field	The secret. Write-only for the web page; readable by the application with `kv_get()`
`place`	Place search by name	The place name. The page also fills the two `number` settings named in `latitude` and `longitude`
`url`	Address of the user's server	Scheme, host and optional port: `http://192.168.1.10:8123`
`link`	Complete URL supplied by the user	`https://host/path?key`, up to 255 bytes. Hidden on the web page like a secret. `webcal://` is converted to `https://`

- `default` is display-only: the page shows it while the setting is empty, and the store stays empty. The code applies the same default when `kv_get()` finds no value.
- `url` and `link` settings are read-only for the application: `kv_set()` on their keys fails. When the user changes such a setting, the host erases the secrets that were not entered again, so a token stays with the server it was entered for.
- `title` is 1 to 31 characters. `help` is up to 95 characters.

6 Screen

Surface and colour

- The origin is the top left corner of the surface. `gfx_width()` and `gfx_height()` return its size. Use these values for layout.
- Drawing calls work inside `app_on_draw()`. The surface is white at the start of each call. In any other handler, `gfx_*` and `ui_*` calls return `HOST_ERR_NOT_DRAWING`. Drawing is clipped to the surface.
- `gfx_color()` sets the colour for the following calls, from `HOST_PAPER` (0) to `HOST_INK` (16). The colour is `HOST_INK` at the start of each `app_on_draw()`. Intermediate values are dithered greys. Use greys for filled areas only: thin lines and small text become unreadable.
- The standard margin is `HOST_UI_MARGIN` (16 px). Use it at the left and right edges.

Fonts

A font constant selects a size. The typeface is a user setting: Departure Mono (monospaced) or Jersey (proportional). Character widths differ between the two, so layout is based on measured widths.

Measure text with `gfx_text_width()`, `gfx_font_height()`, `gfx_font_ascent()` and `gfx_font_cap_height()`.

Constant	Capital height, px	Line height, px	Characters per 400 px	Typical use
`HOST_FONT_SMALL`	8 / 10	15 / 21	57 / 50	Units, captions
`HOST_FONT_TEXT`	16 / 15	30 / 28	28 / 33	Body text; the host's own text
`HOST_FONT_MEDIUM`	24 / 25	45 / 43	19 / 21	Titles, secondary values
`HOST_FONT_LARGE`	32 / 30	60 / 56	14 / 16	Notice lines
`HOST_FONT_DISPLAY`	48 / 50	90 / 86	9 / 10	The main value of a screen
`HOST_FONT_HUGE`	80 / 80	150 / 140	5 / 6	Clock: five digits fill the width

Each cell gives the value for Departure Mono, then for Jersey.

- The `y` argument of `gfx_text()` is the top of the line. The baseline is at `y + ascent`; the top of the capitals is at `y + ascent - cap_height`.
- Add `HOST_FONT_TABULAR` to the font for numbers that change in place, such as a clock or a countdown. All digits then have the same width. Every call that takes a font accepts the flag. Pass the same flag when measuring.
- `gfx_text_fit()` shortens a text to a given width and appends "...". It cuts between characters, so the result is valid UTF-8.

Text beyond ASCII

All text is UTF-8. The host draws each character using the first available option:

1. The glyph of the character. Printable ASCII is always available. Latin letters with diacritics, Cyrillic and Greek are available when the user has installed the glyph pack (Settings, Language, on the device's web page).
2. An ASCII transliteration: "Moskva", "Strasse", "Athina". Typographic dashes and quotes become plain ones.
3. A question mark.

The degree sign is built in at every size.

The width of a text depends on which option was used. Always measure a text before placing it.

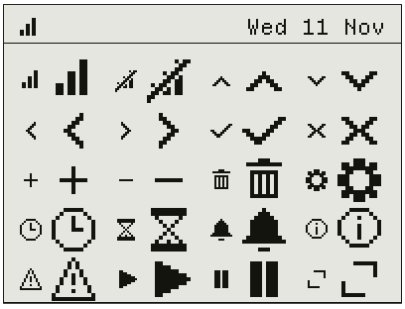
Limits on user-visible text, such as a title or an option, are given in characters. A field holds half as many two-byte characters as one-byte characters.

Pictures and icons

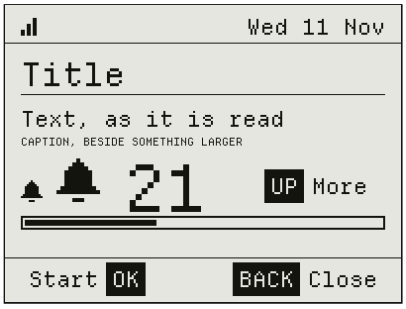
The host provides 20 built-in icons. Each icon is drawn on a grid of 11 x 11 cells. An application can use them on its own screens.

`gfx_icon()` draws an icon with a cell size of 1 to 16 pixels. `ui_icon()` draws an icon at one of three standard sizes: 22, 44 or 99 px.

Constant	Meaning	Constant	Meaning
`HOST_ICON_SIGNAL`	connected	`HOST_ICON_NO_SIGNAL`	not connected
`HOST_ICON_UP`	arrow up	`HOST_ICON_DOWN`	arrow down
`HOST_ICON_LEFT`	arrow left	`HOST_ICON_RIGHT`	arrow right
`HOST_ICON_CHECK`	selected, done	`HOST_ICON_CROSS`	cancel, close
`HOST_ICON_PLUS`	add	`HOST_ICON_MINUS`	remove
`HOST_ICON_TRASH`	delete	`HOST_ICON_GEAR`	settings
`HOST_ICON_CLOCK`	time	`HOST_ICON_HOURGLASS`	waiting, timer
`HOST_ICON_BELL`	notification, alarm	`HOST_ICON_INFO`	information
`HOST_ICON_WARNING`	warning, error	`HOST_ICON_PLAY`	start, play
`HOST_ICON_PAUSE`	pause	`HOST_ICON_UPDATE`	update; the project mark



Built-in icons at cell sizes of 2 and 4 pixels



Basic kit parts: title, text, caption, icons, value, button, bar, button row

An application's own pictures are placed in the `pictures/` folder. Supported formats are PNG, PBM and text. A text file contains rows of `#` (ink) and `.` (paper) and can hold several named pictures:

```
// pictures/things.txt
drop:
.....#.....
....###....
...#####...
..#######..
```

```
#include "pictures.h"   /* generated by mkapp */

gfx_picture(x, y, PICTURE_DROP, 4);    /* 44 px: cell size 4 */
gfx_picture(x, y, PICTURE_DROP, 2);    /* 22 px: cell size 2 */
```

Rules for text files:

- A line of the form `name:` starts a picture.
- Rows before the first such line form a picture named after the file.
- All rows of a picture have the same length.
- A picture is at most 255 cells wide and 255 cells high.

The constant is `PICTURE_` followed by the picture name in upper case. Characters other than letters and digits are replaced with `_`. Example: `half-moon` becomes `PICTURE_HALF_MOON`.

`mkpicture.py --show` prints the pictures as the tool read them.

A picture is stored as one bit per cell. `gfx_picture()` draws it with a cell size of 1 to 16 pixels, so one picture can be used at several sizes.

Use the same grids as the built-in icons: 11 cells, or 22 and 44 cells for more detail. Pictures then match the icons in weight.

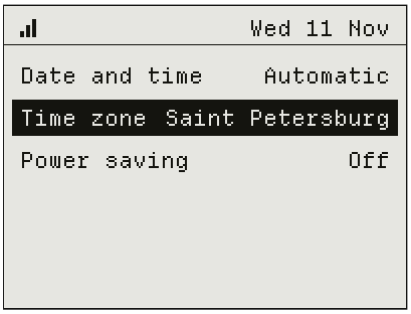
Place a picture that is too large for the module in `files/` and draw it with `gfx_bitmap_file()`. The host reads the file directly; the picture does not use the application's memory.

Kit

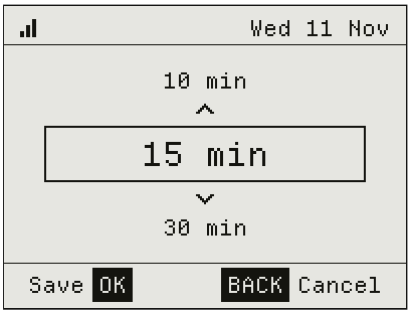
The kit is the set of interface parts that the host's own screens are built from. Applications can use the same parts through the `ui_*` calls.

Using the kit is optional: `gfx_*` calls can draw anything. The kit has three advantages. The application looks like the rest of the system. It follows future changes of the look without a rebuild. It is smaller, because the drawing code is in the host.

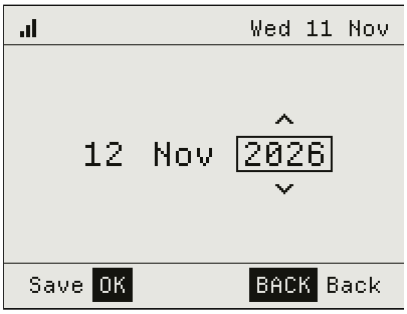
Call	Description	Use
`ui_list_row, ui_list_first, ui_list_visible`	List: one row per line, a value or icon at the right edge, the selected row inverted, a scroll bar. Without a selection, UP and DOWN scroll	More than three items; menus; settings
`ui_cell, ui_cell_fit`	Name, picture and value in a box of any size; the value at the largest size that fits	Dashboards with one, two or four values
`ui_value`	Value with its unit on the same baseline: "18 km/h"	Any number with a unit
`ui_notice`	Two large lines, one text line, a large icon, button labels	Questions, errors, empty states, work in progress
`ui_reason`	Standard notice for a request status: "Loading", "Not set up", "No network", "Access denied", "No answer"	Any screen that depends on the network
`ui_actions, ui_button`	Button row at the bottom: OK action at the left, BACK action at the right	Screens where a button action needs a label
`ui_picker`	One value from a sequence: the selected value framed, neighbours above and below	Selection among a few values with UP and DOWN
`ui_date`	Time, date or both; one part is set at a time	Entry of a moment
`ui_frame`	Frame with arrows for the available directions	Marks the value that the buttons change
`ui_bar`	Filled portion of a total, from the left	Quantities. Changes with the data
`ui_title, ui_text, ui_text_lines, ui_icon`	Page title with a rule; text in four styles, on one line or wrapped; icons at three sizes	Basic page elements



ui_list_row: a list of rows with values



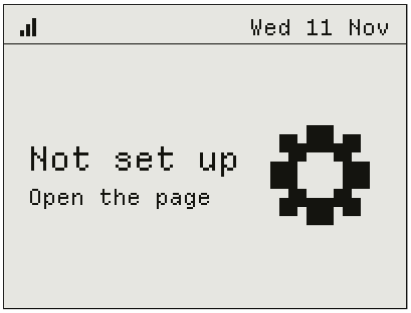
ui_picker: selecting one value



ui_date: setting a date or a time



ui_notice: a question



ui_notice: a message with a reason



ui_notice with is_working: work in progress

Button handling for lists, pickers and dates is implemented in the SDK library (`app_ui.h`). The application calls it from `app_on_button()` and acts on the result. See section 7.

The `kit` application in the catalog shows every part on a separate page. Its source code is a working example for each part.

Requesting a refresh

Flag	Host behaviour	Use
`HOST_REDRAW_NORMAL`	From a button handler: immediate. From a timer, alarm or data handler: when the refresh policy allows, by default once a minute	Default
`HOST_REDRAW_SOON`	Immediate, without a button press	Data the user is waiting for, such as the result of a manual refresh
`HOST_REDRAW_CLEAN`	Full refresh: 2 - 3 s, one flash, removes ghosting	Images that must be clean; the end of a frame sequence
`HOST_REDRAW_NO_BLINK`	Immediate; excluded from the periodic full refresh	List navigation; values that change every second

Redraw requests are unlimited.

Each refresh uses battery energy and wears the panel. Frequent changes also make the device distracting. See section 13 for guidelines.

7 Buttons

The buttons are, in order: UP, OK, DOWN, BACK. The host uses them the same way on every screen. Applications should keep these meanings.

Each press produces two kinds of events. The raw events are `KEYDOWN` and `KEYUP`. The interpreted events are `PRESSED`, sent when a short press ends, and `LONG_PRESSED`, sent after 600 ms while the button is still held. A long press does not produce `PRESSED`.

Most applications handle only `PRESSED` and `LONG_PRESSED`.

The event value combines the button and the action. Each combination has a constant:

```
void app_on_button(int32_t event)
{
    switch (event) {
    case BTN_OK_PRESSED:       start();  break;
    case BTN_OK_LONG_PRESSED:  reset();  break;
    }
    request_redraw(HOST_REDRAW_NORMAL);
}
```

BACK

Application	Short press of BACK
Without `app_on_back()`	Closes the application (one step down the stack)
With `app_on_back()`	Delivered to `app_on_back()`. Typical handling: go one page back; on the first page call `request_exit()`

A short press of BACK goes to `app_on_back()`, or closes the application if it has no such export. A long press of BACK opens the host's menu. `app_on_button()` receives all other events.

`request_exit()` removes the application from the top of the stack. The user returns to the previous screen. This also applies to an application that took the screen with `request_focus()`.

Lists, pickers, dates

```
#include "app_ui.h"

static app_list_t list = { .count = 12 };          /* .selected = -1: no selection, rows scroll */

void app_on_button(int32_t event)
{
    switch (app_list_button(&list, event)) {
    case APP_UI_DONE:      open_item(list.selected);               break;   /* OK */
    case APP_UI_CHANGED:   request_redraw(HOST_REDRAW_NO_BLINK);   break;   /* UP, DOWN */
    default:               break;
    }
}

void app_on_draw(void)
{
    int32_t first = app_list_first(&list);
    for (int32_t row = first; row < first + ui_list_visible(&list) && row < list.count; row++) {
        ui_list_row(&list, row, label_of(row), label_length(row), "", 0, HOST_ICON_NONE);
    }
}
```

`app_picker_t` and `app_date_t` follow the same pattern. The application keeps a struct, passes button events to one function and draws with another. The button function returns *changed*, *done*, *cancelled* or *ignored*.

8 Storage

Store

`kv_get()` and `kv_set()` read and write small values by key. Each application has a private namespace.

The store holds the application's state and its settings. The data is kept when the application is removed and is available again after a reinstall.

```
typedef struct { int32_t version; int32_t minutes; int64_t ends_at; } state_t;
static state_t state;

void app_init(void)
{
    if (kv_get("state", 5, &state, sizeof state) != sizeof state || state.version != 2) {
        state = (state_t){ .version = 2, .minutes = 25 };     /* no stored state, or an older version */
    }
}
```

- Put a version field in every stored struct and check it on load. A newer version of the application will read data written by an older one.
- Every write wears the flash memory. Write only when the data has changed. Do not write on every timer tick.
- `kv_set()` with a length of 0 deletes the key.

Files

Use files for data that does not fit in a store value: a downloaded list, a cache, a picture.

`file_read()` and `file_write()` take an offset. A file can be read and extended in parts, so it never has to be held in memory in full.

The files from the `files/` folder of the package are installed with the application.

9 Network

All network access goes through the host. The application describes a request; the host performs it and delivers the answer. The host performs one request at a time for the whole device. Delivery is independent of whether the application is on the screen.

Network availability is reported in the answer: a request made without a network fails with `HOST_FAILED_NO_NETWORK`.

Method	Trigger	Delivery
Poll with `pick`	Schedule	`app_on_values()`, once, with the extracted values
Poll without `pick`	Schedule	`app_on_data()` in pieces, with the poll number
`http_request()`	Code: after a button press, or as a follow-up request	`app_on_data()` in pieces, with the tag chosen by the application (16 - 255)

Values extracted by the host

```
#include "app_values.h"
enum { VALUE_TEMPERATURE, VALUE_CODE };           /* order of "pick" in the manifest */

void app_on_values(int32_t request, int32_t http_status)
{
    int32_t tenths, code;
    last_status = http_status;                    /* for ui_reason() when there is no data */
    if (http_status == 200 &&
        app_value_number(VALUE_TEMPERATURE, 1, &tenths) &&      /* "21.46" -> 215 */
        app_value_number(VALUE_CODE, 0, &code)) {
        keep(tenths, code);                       /* save to the store */
    }                                             /* otherwise keep the previous data */
    request_redraw(HOST_REDRAW_NORMAL);
}
```

Numbers are integers. `app_value_number()` returns the value scaled by the requested number of decimals, rounded.

A path that is absent from the answer has no value: `app_value_number()` returns false. An absent value means no data.

Streamed answers

```
void app_on_data(int32_t request, const uint8_t *data, int32_t length, int32_t flags, int32_t http_status)
{
    if (flags & HOST_DATA_FAILED) {               /* single call, no data: http_status is a HOST_FAILED_* code */
        last_status = http_status;
        return;
    }
    if (flags & HOST_DATA_FIRST) {                /* the first piece carries the HTTP status */
        last_status = http_status;
        begin_reading();                          /* reset the parser */
    }
    read_piece(data, length);                     /* data is valid until the handler returns */
    if (flags & HOST_DATA_LAST) {
        take_what_was_read();                     /* the answer is complete */
        request_redraw(HOST_REDRAW_NORMAL);
    }
}
```

- Process each piece when it arrives and keep only the data to display. Use a state machine, the SDK's `json_pick` or a custom parser. An answer can be several megabytes; the application has 64 KB.
- Replace the displayed data only on `HOST_DATA_LAST`. Delivery can stop without a last piece and without a failure (section 3). `HOST_DATA_FIRST` always starts a new answer.
- Inside the handler, `data_response()` returns the response metadata: status, Content-Length, Content-Type, ETag, Last-Modified, Location, Link and Retry-After.

Requests from code

```
enum { REQUEST_TOGGLE = HOST_REQUEST_FIRST };
static const char URL[]     = "{{server}}/api/services/light/toggle";
static const char HEADERS[] = "Authorization: Bearer {{token}}\nContent-Type: application/json";

http_request(REQUEST_TOGGLE, HOST_HTTP_POST, URL, sizeof URL - 1, HEADERS, sizeof HEADERS - 1,
             body, body_length);
```

- Requests are allowed only to the hosts listed in the manifest. `{{key}}` placeholders work in the URL and in header values, so the code does not need to read the token.
- `HOST_ERR_NO_SPACE` means that the request queue of the device is full (two waiting requests). Retry later.
- The host follows up to three redirects for a `GET`. A redirect to a host outside the list fails with `HOST_FAILED_NOT_ALLOWED`. A request with a body is sent once; its redirect response is delivered as the answer.
- `poll_now(n)` runs poll `n` immediately, for example when the user presses a refresh button. `poll_set_interval(n, seconds)` changes the interval of a poll from code, for example to honour a Retry-After header.

Failed requests

Status	Cause
`HOST_FAILED_NO_NETWORK (-1)`	No network connection
`HOST_FAILED_NO_ANSWER (-2)`	No answer from the server, or an incomplete answer
`HOST_FAILED_NOT_READY (-3)`	A `{{setting}}` used by the request is empty
`HOST_FAILED_NOT_ALLOWED (-4)`	Redirect to a host outside the manifest's list
`HOST_FAILED_TIMEOUT (-5)`	Timeout after the server was reached
`HOST_FAILED_NOT_TRUSTED (-6)`	HTTPS certificate is not trusted by the device
`HOST_FAILED_TOO_LARGE (-7)`	Answer larger than 256 KB (extracted values) or 4 MB (streamed)
`HOST_FAILED_NO_MEMORY (-8)`	Insufficient device memory for the request

Save the last status in the store. When there is no data to display, pass the status to `ui_reason()`. It draws the standard notice for the status and returns whether it drew anything:

Status	Notice	Return value
`0 (no request yet)`	Loading	1
`HOST_FAILED_*`	One notice per code: Not set up, No network, No answer ...	1
`401, 403`	Access denied / Check the settings	1
`404`	Not found / Check the address	1
`Any other non-2xx status`	No answer / Code <status>	1
`200 – 299`	None. The answer arrived; missing data is reported by the application	0

10 Background work

The host loads an off-screen application for a single call when one of these events occurs:

Task	Mechanism	Notes
Scheduled data	Poll in the manifest	Active after a restart without code
Action at a set time	`alarm_set, alarm_set_at, alarm_set_every`	Alarms are cleared by a restart. Restore them in the `started` intent from a time kept in the store
Message to the user	`notify`	Shown by the host over the current screen until dismissed
Bring the application to the screen	`request_focus`	Granted by the host, at most once per 30 s
Status in the header	`header_status`	An icon and a short text in the header above every screen while the application is off the screen
Reaction to device start, network connection or clock change	Intent in the manifest	`app_on_intent()`

Notifications

```
void app_on_alarm(int32_t id)                    /* the application is loaded for this call */
{
    host_notice_t notice = { .first = "Time", .second = "is up", .icon = HOST_ICON_BELL };
    notify(&notice);
}
```

After `notify()` returns, the host manages the notification. It shows the notice under the header, keeps it until the user presses OK or BACK, and then restores the previous screen.

The notification is managed entirely by the host. The application stays unloaded while it is displayed.

Use `notify()` instead of `request_focus()` when the message fits in two lines.

Alarms, timers

Call	Behaviour
`timer_set(id, milliseconds)`	One shot. Calls `app_on_timer(id)`. Active only while the application is on the screen; set again in `app_on_focus()`
`alarm_set(id, seconds)`	One shot. Calls `app_on_alarm(id)` on or off the screen. Measures elapsed time, independent of clock changes
`alarm_set_at(id, unix_time)`	One shot at a clock time. Follows clock changes
`alarm_set_every(id, seconds, first_in)`	Repeating, every 60 s to 1 day. Each call loads the application; use the longest suitable period

11 Time

- `time_now()` returns Unix time in seconds. It returns 0 until the device has obtained the time. The application must handle this case.
- Display local time. The host owns the time zone. `time_local()` returns the current local date and time, `time_local_at()` converts any moment, and `time_from_local()` converts a local date and time back to Unix time, for example for `alarm_set_at()`.
- `time_text()` formats a moment as text in the device's language. Applications do not need their own names of days and months. Formats: "15:04", "Fri 13 Nov", "Friday", "13 November", and a relative form: "15:04", "Tmrw 15:04", "Sat 15:04", "21 Nov".
- To be called when the clock is set or changed, declare the `time_changed` intent.

12 Languages

An application displays its text in the language of the device. The developer provides the translations. The host loads the translation for the current language.

```
strings/en.json    { "no_events": "No events", "starts_in": "In {0} min" }
strings/ru.json    { "no_events": "Нет событий", "starts_in": "Через {0} мин",
                     "manifest": { "title": "Календарь",
                                   "settings": { "every": { "title": "Обновление",
                                                            "options": { "15 min": "15 мин" } } } } }
```

```
#include "strings.h"                             /* generated by mkapp: STR_NO_EVENTS, STR_STARTS_IN, app_string() */

const char *text = app_string(STR_NO_EVENTS);    /* text in the device's language */

const host_text_argument_t minutes = { .number = 10 };
char line[32];
int32_t length = text_fill(app_string(STR_STARTS_IN), app_text_length(app_string(STR_STARTS_IN)),
                           &minutes, 1, line, sizeof line);          /* "In 10 min" */
```

- `en.json` is required. English is compiled into the module and is used when a translation is unavailable. Every other file contains exactly the same keys. The key `manifest` is allowed in translations only (see below).
- When the host loads the application, it copies the string table of the device's language into the application's memory, over the English table. `app_string()` then returns the translated text with no extra call.
- `text_fill()` replaces `{0}`, `{1}` in a string with numbers or texts. A translation can place them in any order. Write each string in one form for all numbers, for example "min".
- The `manifest` key of a translation contains the translated parts of the manifest: the title, the title and help of each setting, and labels for the options of a choice. The stored option value does not change; only its label is translated.
- The build fails if a language is not listed in `sdk/languages.json` (27 languages, two-letter codes), if a translation has a missing or an extra key, or if a string contains a character that the glyph packs of the language do not cover.
- Translations are usually about a third longer than English. The kit shortens text that does not fit with "...". Check the translated screens on the device.

13 Design rules

These rules apply to all applications. The host's own screens follow them. Most are conventions; the host checks only a few.

Screen content

Show one main value per screen.
The device is read from about a metre, in about a second. Choose the main value, draw it largest, and keep everything else small.

Keep screens static.
Change a screen only when its data changes. A bar shows a quantity. To indicate work in progress, show a notice with `is_working`.

Redraw only when the content changes.
A timer should request a redraw only if the picture will differ. A clock redraws once a minute, on the minute. By default the host allows one refresh per minute that was not caused by a button press. Design for this rate.

Draw black on white.
Use ink on paper. Use grey for filled areas only. Use white text on a black bar only for the selected item or for new information.

Keep interface text short and standard.
Use common words: "Save", "Remove", "Not set up". If a label needs an explanation, change the label.

Memory

Minimize code size before data size.
Each byte of executed code uses about four bytes of the interpreter's 64 KB. Each byte of data uses one byte of the 64 KB page, which is mostly free. Prefer a table to a chain of `if` statements, the kit to custom drawing, `pick` to a custom parser, and `time_text()` to custom date formatting.

Keep shared helpers out of line.
Mark a small helper that is called from many places with `__attribute__((noinline))` and compare the build reports.

Store pictures as bits.
Put pictures in `pictures/`. Put large pictures in `files/`.

Process answers as a stream.
Process an answer as it arrives and keep only the data to display. Allocate working buffers with `app_alloc()` and release them with a mark.

State and time

Save state before every unload.
All state that matters must be in the store when `app_on_terminate()` returns. To test: open the application, leave it, open it again. The screen must look the same.

Keep the displayed data until new data is complete.
After a failed or interrupted request, keep the previous data on the screen.

Explain an empty screen.
Call `ui_reason()` to show "Not set up", "No network" or "Loading".

Handle an unknown or changed clock.
Handle `time_now() == 0`. Use `alarm_set()` for a delay ("in ten minutes") and `alarm_set_at()` for a clock time ("at seven").

Interaction and data

Take text input on the web page.
Declare text such as a token, an address or a city as a manifest setting; the user enters it on the device's web page. On the device, let the user choose from a few values with UP and DOWN.

Take the screen rarely.
Use a notification only for events the user asked to be told about, such as a timer or a meeting. Let other information wait until the user looks.

Send user data only where the user directs.
List only the hosts the application needs. Use a `url` setting for the user's own server. Use a `secret` setting for a token and send it through a header placeholder.

Poll no more often than the data changes.
Offer the interval as a setting with a reasonable default. On battery power the host limits every poll to one per five minutes.

14 Tools and debugging

Command	Function
`tools/mkapp/mkapp.py <dir> --out <dir>`	Builds one application: compile, link with the SDK library, check, measure
`tools/devtool/devtool.py new <dir> [--title T]`	Creates a new application folder from the SDK sample
`tools/devtool/devtool.py run <dir> [--shot f.png]`	Builds, uploads and restarts the device into the application
`tools/devtool/devtool.py serve [dir]`	Local web page: live screen, buttons, log, Run
`tools/devtool/devtool.py shot [f.png]`	Saves the current panel frame as an image
`tools/devtool/devtool.py press 3 3 2L [--shot f.png]`	Sends button presses: 1 - 4 short, 1L - 4L long, in device order
`tools/devtool/devtool.py open <name>`	Opens an installed application
`tools/devtool/devtool.py log [--follow] [--app]`	Prints the device log. `--app`: application lines only
`tools/devtool/devtool.py set <app> <key> <value>`	Writes a value to an application's store, for example a setting
`tools/devtool/devtool.py status`	Prints firmware, network, free memory and the last reset reason
`tools/mkpicture/mkpicture.py <files> --show`	Prints pictures as rows of # and . as `mkapp` reads them
`tools/mkvideo/mkvideo.py <video> --out <bin>`	Converts a video to a 1-bit stream that fits in an application file
`tools/catalog/mkcatalog.py <folder> --out <dir>`	Builds a folder of applications into a catalog

`host_log(level, text, length)` writes one line of up to 200 characters to the device log. It replaces `printf`.

`press ... --shot` waits for the panel to finish and saves the frame. This can be used to script regression tests of the application's screens.

Common problems

Symptom	Probable cause
Blank screen	Drawing outside `app_on_draw()`, or a missing `request_redraw()` after a state change
State lost after leaving and returning	State kept in global variables. Save it in `app_on_terminate()`, load it in `app_init()`

Symptom	Probable cause
"<title> crashed" with "memory alloca..."	Executed code exceeds the interpreter memory. See the build warning and "Memory" in section 13
"<title> cannot start", "Update the app."	Built against a different `host_api.h` than the device firmware. Rebuild and reinstall
Poll fails with -3	A `{{setting}}` in its URL or headers is empty
HTTPS fails with -6	Self-signed or expired server certificate. Use `http` within a home network
`http_request()` returns -4	The device request queue is full. Retry from a later handler
Ghost images of earlier frames	A sequence of `NO_BLINK` redraws without a final `CLEAN` redraw
Characters shown as ? or transliterated	The glyph pack for the script is not installed, or the text was truncated by bytes
A press is ignored shortly after start	A background delivery was in progress. The press interrupted it and the screen is being redrawn

15 Publishing

A device installs applications from *catalogs*. A catalog is a folder of static files with an index. It can be hosted anywhere that serves HTTPS, for example as raw files of a git repository or as a static site.

```
tools/catalog/mkcatalog.py my-apps --out catalog       # build all applications in the folder
                                                       # output: catalog/catalog.json, one folder per application
```

- The index lists the name, title, version and icon of each application, and its files with their sizes and SHA-256 hashes. The device accepts a file only if both the size and the hash match.
- Trust model: HTTPS to a source that the user selected, plus the sandbox. Packages are unsigned.
- The official catalog is the `perinkpheral-apps` repository. It is the first source on every device. The user can add up to three more sources on the device's web page by entering a repository address.
- The web page shows a summary of each installed application, taken from its manifest: the servers it connects to, the polling interval, what it stores, and whether it starts with the device.
- Increase `version` with every change. The device compares versions to detect updates.

Part III. Reference

This part lists every declaration of `sdk/host_api.h` for ABI 2.

Numeric arguments and return values are `int32_t` unless stated otherwise. A text argument is a pointer and a length in bytes; the text is UTF-8.

A call that returns no value of its own returns `HOST_OK` or a negative `HOST_ERR_*` code.

16 Return codes

Code	Meaning
`HOST_OK (0)`	Success
`HOST_ERR_NOT_DRAWING (-1)`	Drawing call outside `app_on_draw()`
`HOST_ERR_ARGUMENT (-2)`	Pointer outside the application's memory; unknown font, icon, timer or style; size out of range
`HOST_ERR_NOT_FOUND (-3)`	Key not stored; file not found; time not set; path absent from the answer
`HOST_ERR_NO_SPACE (-4)`	Value larger than the buffer; store or files full; request or notification queue full
`HOST_ERR_FAILED (-5)`	Storage failure

17 Exports

All exports are optional. The host skips a handler that the module does not export.

Export	Called when
`void app_init(void)`	The instance is loaded and its strings are in place. Once per instance
`void app_on_focus(void)`	The application receives the screen. Start `timer_set()` timers here
`void app_on_blur(void)`	The application is about to lose the screen
`void app_on_terminate(void)`	The instance is about to be unloaded, for any reason: leaving the screen, end of a background call, device restart, removal. `app_on_blur()` precedes it if the application was on the screen
`void app_on_draw(void)`	A frame is needed. Draw the complete frame from the state. The surface is white
`void app_on_button(int32_t event)`	A button event occurs: `BTN_<UP|OK|DOWN|BACK>_<KEYDOWN|KEYUP|PRESSED|LONG_PRESSED>`. Excludes `BTN_BACK_PRESSED` and `BTN_BACK_LONG_PRESSED`
`void app_on_back(void)`	BACK is pressed briefly. For applications with internal pages
`void app_on_timer(int32_t id)`	A timer expires
`void app_on_alarm(int32_t id)`	An alarm expires, on or off the screen
`void app_on_data(int32_t request, const uint8_t *data, int32_t length, int32_t flags, int32_t http_status)`	A piece of an answer arrives. `request` is the poll number or the request tag. `flags`: `HOST_DATA_FIRST`, `_LAST`, `_FAILED`

Export	Called when
`void app_on_values(int32_t request, int32_t http_status)`	The answer to a poll with `pick` arrives. Values are readable inside the handler
`void app_on_settings(void)`	The user saves the application's settings on the web page
`void app_on_intent(int32_t intent)`	A declared device event occurs: `HOST_INTENT_TIME_CHANGED` (1), `_NETWORK_UP` (2), `_STARTED` (3)
`uint8_t *app_data_buffer(void)`	Returns the buffer for one piece of data (1 KB). Provided by `sdk/lib`
`uint8_t *app_strings_buffer(void)`	Returns the buffer for the strings of the device's language. Generated by `mkapp`

18 Drawing

Call	Description
`gfx_width(void), gfx_height(void)`	Surface size: 400 x 260 below the header, 400 x 300 full-screen
`gfx_color(color)`	Sets the drawing colour: `HOST_PAPER` (0) to `HOST_INK` (16)
`gfx_text(x, y, font, text, length)`	Draws text; y is the top of the line. Returns the width. Up to 127 bytes
`gfx_text_width(font, text, length)`	Returns the width of the text
`gfx_text_fit(font, text, length, max_width, fitted, capacity)`	Shortens the text with "..." to fit the width. Returns the new length
`gfx_font_height(font), gfx_font_ascent(font), gfx_font_cap_height(font)`	Line height; distance from the line top to the baseline; height of capitals and digits
`gfx_rect(x, y, w, h, fill)`	Rectangle: `HOST_RECT_OUTLINE` or `HOST_RECT_FILLED`
`gfx_line(x0, y0, x1, y1)`	Line, 1 px wide, end points included
`gfx_picture(x, y, picture, cell)`	Picture generated by `mkapp` from `pictures/`: width byte, height byte, one bit per cell. Cell size 1 - 16 px
`gfx_icon(icon, x, y, cell)`	Built-in icon `HOST_ICON_*`, 11 x 11 cells. Cell size 1 - 16 px
`gfx_bitmap(x, y, w, h, bits)`	Bitmap, one bit per pixel. Set bits are drawn in the current colour; clear bits are transparent
`gfx_bitmap_file(x, y, w, h, name, name_length, offset)`	Bitmap read by the host from an application file, starting at `offset`
`gfx_qr(x, y, cell, text, length)`	QR code of up to 200 bytes. Returns the number of modules per side. Cell 0: measure only
`void request_redraw(flags)`	Requests a redraw. Flags: `HOST_REDRAW_NORMAL`, `_SOON`, `_CLEAN`, `_NO_BLINK`
`header_text(side, text, length)`	Sets the application's text in the host header above its screen: `HOST_HEADER_LEFT` or `_RIGHT`, up to 63 bytes. Length 0 returns the side to the host. Callable from any handler

19 Kit

Call	Description
`ui_text(x, y, style, text, length, max_width)`	Text in a style: `HOST_UI_TEXT`, `_TITLE`, `_CAPTION`, `_VALUE`; shortened to `max_width` (0: unlimited). Returns the width
`ui_text_lines(x, y, style, text, length, width, max_lines)`	Text wrapped at spaces to `width`, up to `max_lines` (1 - 8); the last line is shortened with "...". Returns the number of lines
`ui_text_width(style, text, length), ui_text_height(style)`	Text width and line height of a style

Call	Description
`ui_title(text, length)`	Page title with a rule. Returns the y of the content area
`ui_icon(x, y, icon, size)`	Built-in icon at `HOST_UI_ICON_SMALL` (22 px), `_MEDIUM` (44), `_LARGE` (99)
`ui_actions(ok, ok_length, back, back_length)`	Button row at the bottom, 44 px (`HOST_UI_ACTIONS_HEIGHT`). Length 0 omits a button; a single button is centred
`ui_button(x, y, name, name_length, action, action_length, mirrored)`	One button: its name on a plate and the action label beside it. Returns the width
`ui_list_visible(list), ui_list_first(list)`	For a `host_list_t` (count, selected or -1, top, height or 0 for the remaining height, first): number of visible rows; index of the first visible row
`ui_list_row(list, index, label, label_length, value, value_length, icon)`	One row. Rows outside the visible range are skipped. A value up to half the row width is kept whole; the label is shortened with "..."
`ui_notice(notice)`	Notice on the whole surface from a `host_notice_t`: first[64], second[64], detail[96], ok[32], back[32], icon, is_working. Empty fields are omitted
`ui_reason(status)`	Standard notice for a request status. Returns 1 if drawn, 0 for a 2xx status
`ui_value(x, y, font, value, value_length, unit, unit_length)`	Value with its unit on the baseline. Returns the total width
`ui_value_width(font, value, value_length, unit, unit_length)`	Width of a value with its unit
`ui_cell(cell, picture)`	Cell from a `host_cell_t` (x, y, width, height, name[48], value[48], size) with an optional picture (0: none). Picture size: 88 px in a full-surface cell, 44 px otherwise
`ui_cell_fit(cell, picture)`	Returns the size class that fits the cell; 1 is the largest. Use the largest returned number for all cells of a screen
`ui_bar(x, y, width, value, maximum)`	Bar, 14 px high: `value` of `maximum`
`ui_frame(x, y, width, height, arrows)`	Frame with optional arrows: `HOST_UI_ARROW_UP`, `_DOWN`
`ui_picker(picker)`	Picker from a `host_picker_t`: above[64], chosen[64], below[64], ok[32], back[32]. Includes the button row
`ui_date(date)`	Date and time editor from a `host_date_t`: year, month, day, hour, minute; has (`HOST_DATE_HAS_TIME | _DATE`); selected (`HOST_DATE_HOUR .. _YEAR`, or `_NONE`); ok[32], back[32]

20 Navigation, storage and network

Call	Description
`void request_exit(void)`	Closes the application: one step down the stack. Valid in button, back and timer handlers
`void request_focus(void)`	Requests the screen from a background handler. Granted by the host, at most once per 30 s
`notify(notice)`	Queues a notification from a `host_notice_t`: first (required), second, detail, icon. `HOST_ERR_NO_SPACE` when four are queued
`header_status(side, icon, text, length)`	Sets an icon and up to 31 bytes of text in the header, shown above every screen while the application is off the screen. Empty icon and text release the side. Cleared by a restart
`timer_set(id, milliseconds)`	One-shot timer, id 0 - 3. 0 ms cancels. A new call restarts it
`alarm_set(id, seconds)`	One-shot alarm, id 0 - 3. 0 s cancels
`alarm_set_at(id, int64_t unix_time)`	One-shot alarm at a clock time
`alarm_set_every(id, seconds, first_in)`	Repeating alarm, period 60 - 86400 s. First call after `first_in` seconds (0: one period)

Call	Description
`kv_get(key, key_length, value, capacity)`	Reads a value. Returns its length; `HOST_ERR_NOT_FOUND`; `HOST_ERR_NO_SPACE` if the buffer is too small
`kv_set(key, key_length, value, length)`	Writes a value. Length 0 deletes the key. Fails for `url` and `link` settings
`file_size(name, name_length)`	Returns the file size, or `HOST_ERR_NOT_FOUND`
`file_read(name, name_length, offset, bytes, capacity)`	Reads from an offset. Returns the number of bytes read; 0 at or past the end
`file_write(name, name_length, offset, bytes, length)`	Writes at an offset within the file or at its end. Length 0 at offset 0 deletes the file
`poll_now(poll)`	Runs a manifest poll immediately
`poll_set_interval(poll, seconds)`	Sets the poll interval, 10 s to 1 day. Kept across restarts
`http_request(tag, method, url, url_length, headers, headers_length, body, body_length)`	Queues a request. Methods: `HOST_HTTP_GET`, `_POST`, `_PUT`, `_DELETE`, `_PATCH`, `_HEAD`, `_PROPFIND`, `_REPORT`. Headers: "Name: value" lines
`data_value(index, text, capacity)`	Inside `app_on_values()`: copies the value of path `index` as text without a terminator. Returns its length
`data_response(response)`	Inside a data handler: fills a `host_response_t`: status, length, retry_after, content_type[48], etag[64], last_modified[32], location[128], link[192]
`text_fill(pattern, pattern_length, arguments, count, text, capacity)`	Replaces `{0}` .. `{3}` in the pattern with `host_text_argument_t` values (a number, or a text and its length). Returns the length
`void host_log(level, text, length)`	Writes one log line of up to 200 characters. Levels: `HOST_LOG_ERROR`, `_WARN`, `_INFO`, `_DEBUG`

21 Time

Call	Description
`int64_t time_now(void)`	Unix time in seconds; 0 while the time is unknown
`time_zone_offset(void)`	Offset of local time from UTC in seconds, daylight saving included
`time_local(parts), time_local_at(int64_t unix_time, parts)`	Fills a `host_local_time_t`: year, month 1 - 12, day, weekday 0 = Sunday, hour, minute, second
`int64_t time_from_local(parts)`	Unix time of a local date and time. Out-of-range days carry into the next month
`time_text(int64_t unix_time, style, text, capacity)`	Formats a moment in the device's language. Styles: `HOST_TIME_CLOCK`, `_DATE`, `_WEEKDAY`, `_LONG_DATE`, `_NEAR`, `_NEAR_DAY`. Returns the length
`int64_t time_uptime_ms(void)`	Milliseconds since device start; includes sleep time
`uint32_t time_boot_id(void)`	Identifier of the current boot; changes after every restart, never 0. Store it with a saved uptime

22 System API

The system API is the set of `sys_*` calls. The host's own pages use it.

Any application can use the system API if its manifest contains `"system": true`. The web page warns the user before installing such an application. A module that imports a `sys_*` call without this declaration is rejected.

Call	Description
`sys_info(what, text, capacity)`	Device information as text: `SYS_INFO_VERSION`, `_NAME`, `_NETWORK`, `_ADDRESS`, `_WIFI` ("online", "connecting", "setup"), `_SLOT`, `_FREE_MEMORY`, `_TIME_ZONE`, `_JOIN_CODE`
`sys_number_get(what, number)`, `sys_number_set(what, value)`	Numeric host setting, range-checked: `SYS_NUMBER_TIME_AUTOMATIC`, `_REFRESH_FULL_EVERY`, `_REFRESH_MIN_SECONDS`, `_REFRESH_FORCE_HOURS`, `_POWER_SAVING`, `_REFRESH_FAST`, `_FONT`, `_MENU_IDLE_MINUTES`, `_HEADER_CLOCK`
`sys_text_set(what, text, length)`	Text host setting: `SYS_TEXT_TIME_ZONE`, a POSIX TZ string
`sys_time_set(int64_t unix_seconds)`	Sets the clock. Fails while the time source is the network
`sys_app_list(ids, capacity)`, `sys_app_title(id, text, capacity)`, `sys_app_remove(id)`	Installed applications by id, up to 16: list, title, removal
`sys_app_default(void)`, `sys_app_set_default(id)`	Default application id; 0: none
`sys_logo(x, y, scale)`, `sys_logo_width(scale)`, `sys_logo_height(scale)`	The wordmark perINKpheraL in square letters, scale 1 - 5: draw, width, height
`sys_do(action)`	`SYS_DO_RESTART`, `_FORGET_NETWORK`, `_FACTORY_RESET`. Requires a confirmation screen in the application

23 SDK library

`mkapp` links the library into every application. Unused functions are removed. The headers are in `sdk/lib`.

`bool` is the standard type from `<stdbool.h>`. Text arguments are null-terminated unless stated otherwise.

app_text.h: short strings without printf

```
typedef struct { char *next; char *last; } app_text_t;      /* declared by the caller */

void    app_text_start(app_text_t *text, char *buffer, int32_t buffer_size);
void    app_text_add(app_text_t *text, const char *more);   /* excess text is dropped at a character boundary */
void    app_text_add_number(app_text_t *text, int32_t value);               /* decimal, "-" when negative */
void    app_text_add_padded(app_text_t *text, int32_t value, int32_t digits);          /* 7, 2 -> "07" */
int32_t app_text_length(const char *text);                  /* length of any null-terminated text */
```

The buffer is always null-terminated and stays within its size. The text is complete after each call.

app_values.h: what the host picked

```
bool app_value_text(int32_t index, char *text, int32_t text_size);        /* false: value absent or buffer too small */
bool app_value_number(int32_t index, int32_t decimals, int32_t *number);  /* "-3.46", 1 -> -35; false: value absent */
bool app_parse_number(const char *text, int32_t decimals, int32_t *number);    /* the same for a given text */
```

These functions work only inside `app_on_values()`. `index` is the position of the path in the `pick` list of the manifest.

app_ui.h: what the buttons do on the kit's screens

```
typedef enum { APP_UI_IGNORED, APP_UI_CHANGED, APP_UI_DONE, APP_UI_CANCELLED } app_ui_result_t;

typedef host_list_t app_list_t;        /* count, selected (-1: none), top, height, first */
app_ui_result_t app_list_button(app_list_t *list, int32_t event);    /* UP, DOWN: changed; OK: done; BACK: cancelled */
int32_t         app_list_first(app_list_t *list);                    /* call in app_on_draw() before drawing rows */

typedef struct { int32_t count; int32_t chosen; bool wraps; } app_picker_t;
app_ui_result_t app_picker_button(app_picker_t *picker, int32_t event);
void            app_picker_draw(const app_picker_t *picker, const char *const *labels);

typedef struct { int32_t year, month, day, hour, minute, has, part, year_first, year_last; } app_date_t;
void            app_date_start(app_date_t *date, int32_t has, int32_t year_first, int32_t year_last);
app_ui_result_t app_date_button(app_date_t *date, int32_t event);    /* OK: next part; done after the last part */
void            app_date_draw(const app_date_t *date);
int64_t         app_date_seconds(const app_date_t *date);            /* local time; subtract time_zone_offset() for UTC */
```

To handle BACK on these screens, export `app_on_back()` and pass `BTN_BACK_PRESSED` from it to the same button function.

app_alloc.h, json_pick.h, and what is generated

```
void   *app_alloc(int32_t bytes);           /* NULL if the arena is too small; memory is not zeroed */
int32_t app_alloc_left(void);
int32_t app_alloc_mark(void);               /* app_alloc_release(mark) frees everything allocated after the mark */
void    app_alloc_release(int32_t mark);

void        json_pick_init(json_pick_t *pick, const char *const *paths, int path_count);    /* up to 16 */
bool        json_pick_feed(json_pick_t *pick, const char *bytes, size_t length);      /* one piece of the answer */
void        json_pick_finish(json_pick_t *pick);
const char *json_pick_value(const json_pick_t *pick, int index);         /* NULL: value absent */

#include "strings.h"      /* generated from strings/:  enum { STR_NO_EVENTS, ... };  const char *app_string(int32_t id); */
#include "pictures.h"     /* generated from pictures/: static const uint8_t PICTURE_DROP[] = { width, height, bits }; */
```

`strings.h` and `pictures.h` are generated for each application. The build places them first on the include path, so a system header with the same name is not used.

`memcpy`, `memmove`, `memset` and `strlen` are provided because the compiler generates calls to them. They are the whole C library of the platform.

24 Limits

Constant	Value	Applies to
`HOST_API_VERSION`	2	`abi` in the manifest
`MANIFEST_MODULE_MAX_BYTES`	48 KB	`app.wasm`
`MANIFEST_NAME_MAX / _TITLE_MAX`	13 / 23 characters	Application name and title
`MANIFEST_POLLS_MAX / _HOSTS_MAX / _SETTINGS_MAX`	4 / 4 / 8	Polls, hosts and settings per manifest
`MANIFEST_URL_MAX / _SETTING_VALUE_MAX`	511 / 255 bytes	Poll URL; setting value
`HOST_TEXT_MAX`	127 bytes	Text per drawing or measuring call
`HOST_HEADER_TEXT_MAX / HOST_STATUS_TEXT_MAX`	63 / 31 bytes	Header text; header status text
`HOST_KV_KEY_MAX / _VALUE_MAX / _TOTAL_MAX`	15 / 1024 / 4096	Store key; value; total per application
`HOST_FILE_NAME_MAX / HOST_FILES_MAX`	23 / 64 KB	File name; total file size per application
`HOST_DATA_CHUNK_MAX`	1024 bytes	One piece of an answer
`HOST_ANSWER_MAX / HOST_ANSWER_PIECES_MAX`	256 KB / 4 MB	Answer with extracted values; streamed answer
`HOST_VALUE_MAX`	63 bytes	One extracted value

Constant	Value	Applies to
`HOST_HTTP_HEADERS_MAX / _BODY_MAX / _PENDING_MAX`	1023 / 512 / 2	Request headers; body; queued requests per device
`HOST_REQUEST_FIRST .. _LAST`	16 .. 255	Tags for requests from code
`HOST_TIMER_COUNT / HOST_ALARM_COUNT`	4 / 4	Timers and alarms per application
`HOST_ALARM_EVERY_MIN .. _MAX`	60 .. 86400 s	Period of a repeating alarm
`HOST_TEXT_ARGUMENTS_MAX`	4	Arguments of `text_fill()`
`HOST_QR_TEXT_MAX / HOST_LOG_TEXT_MAX`	200 / 200	QR code text; log line
`HOST_ICON_CELLS / HOST_ICON_CELL_MAX`	11 / 16	Icon grid; maximum cell size in pixels
`String table of one language`	8 KB	Loaded by the host

`sdk/host_api.h` is the authoritative contract. If this document and the header disagree, the header is correct.